

| IEEE STUDENT BRANCH · VIT BHOPAL

BUILD BEYOND BOUNDARIES

Problem Statements Dossier

OFFICIAL RELEASE · 27 SEP

REGISTRATIONS CLOSED

5 TRACKS · 9 STATEMENTS

IDEA DECKS DUE SAT 03 OCT

GRAND SHOWCASE SAT 10 OCT

PARTICIPANT GUIDELINES

GUIDELINES & SUBMISSION TENETS

Modern AI Engineering · Scope Boundary · Submission Rules

SCOPE CHECK

Focus on one clear use case and demonstrate a working solution. You do not need to cover every aspect mentioned in the problem statement. A well-executed, focused prototype that clearly demonstrates your core idea is enough.

AI Policy: Full Permission with Technical Ownership

You are **fully permitted** to use modern AI systems throughout this hackathon: foundation models (ChatGPT, Claude, Gemini, DeepSeek, Qwen, etc.) and agentic developer tools (Antigravity, Cursor, Windsurf, Copilot, OpenCode, DeepSeek Harness, v0, Bolt, etc./others). Modern engineering leverages AI. Your team will be evaluated on technical ownership:

- **Understand & Defend Decisions:** You must understand every architectural choice, circuit layout, or algorithm and defend it during evaluation.
- **Technical Depth:** Strive for authentic problem-solution fit where AI acts as a development accelerator rather than an uninspected black box.
- **Human Orchestrator:** Act as the engineer directing the tools. Test and verify all generated code; never blindly commit unverified outputs.
- **Responsible Technology:** Build ethically and understand the impact of your technology. Secure permission for data and test on sanitized datasets.

CORE PARTICIPATION TENET · AUTHENTIC PROBLEM SOLVING

Every brief describes an authentic engineering challenge. Two teams on the same brief must develop completely distinct solutions based on their unique architecture and user focus.

Idea Deck Submission Guidelines · Screening Round

STRICT 5-7 SLIDES · PPTX / PDF

Submit a concise deck detailing: (1) **Problem & Core Idea**, (2) **Expected Methodology & Architecture**, and (3) **Tooling & Stack**. Teams with early prototypes or simulations are welcome to include demos (optional for idea round).

SUBMISSION DUE · SATURDAY 03 OCT, 11:59 PM IST

Official Template Provided

Both .pptx and .pdf formats are permitted. You may build upon or export from the official Idea Deck template provided.

Universal Evaluation Tenets — What Every Team Should Aim to Demonstrate

- **Understand the Problem:** Clearly explain what problem you are solving, who it is for, and why it matters.
- **Show How It Works:** Demonstrate the main features or workflow of your solution (does not need to be a finished product).
- **Know Your Work:** Everyone on the team should be able to explain how the solution works and defend technical choices made.
- **Build a Solution:** Present a working prototype, model, design, or simulation that addresses the problem.
- **Test Your Idea:** Show evidence that your solution works (test cases, results, comparisons, or user feedback).

Philosophy: You are not expected to build a production-ready product. A well-executed, focused prototype that clearly demonstrates your idea is enough. Focus on learning, building, testing, and explaining your solution.

TRACK DIRECTORY & PROBLEM INDEX

Master Navigation · 5 Engineering Tracks · 9 Released Problem Statements

TRACK	DOMAIN SCOPE	CODE	OFFICIAL PROBLEM STATEMENT TITLE
T01 Hardware	Robotics · Edge AI · Assistive Tech · Smart IoT Systems	01.1	Open Bench: build what you like
T02 UI / UX	AI Interfaces · Accessibility · Human-Centred Redesign	02.1	The Process Everyone Dreads
T02 UI / UX	AI Interfaces · Accessibility · Human-Centred Redesign	02.2	The Living Canvas: Human-AI Collaboration
T03 Gen AI & ML	Computer Vision & Detection · Applied ML · Neural Optimization & Agents	03.1	Vision in the Wild: Applied Machine Learning & Detection
T03 Gen AI & ML	Computer Vision & Detection · Applied ML · Neural Optimization & Agents	03.2	The Learned Loop: From Search to Learning
T04 Cybersecurity	FinTech · Fraud Shield · Zero-Trust Cybersecurity	04.1	The Scam That Almost Worked
T04 Cybersecurity	FinTech · Fraud Shield · Zero-Trust Cybersecurity	04.2	Shipped Fast, Left Open
T05 Sustainable Living	Climate Tech · Corporate Sustainability · Resource Efficiency · Smart Infrastructure	05.1	When the Grid Trembles: Carbon-Aware Infrastructure
T05 Sustainable Living	Climate Tech · Corporate Sustainability · Resource Efficiency · Smart Infrastructure	05.2	Hyperlocal Climate Resilience: Heat, Drought, and Weather Extremes

| TRACK 01 · PHYSICAL SYSTEMS

TRACK 01 · HARDWARE

Robotics · Edge AI · Assistive Tech · Smart IoT Systems

Open Bench: build what you like

01.1

COVERS: Robotics, edge computing, assistive devices, embedded systems, VLSI, instrumentation, power

- Build any hardware project, provided it relates to one of the hackathon themes: **the future, security or sustainability**. This includes technical fields not explicitly named in the track title: electronics, embedded systems, VLSI, instrumentation, and power engineering.
- You may continue existing work from a previous semester, a laboratory rig, or an unfinished board. Bring the project, add one distinct improvement during the build window, and state clearly what was built before the hackathon and what was created during it.
- Working physical prototypes are preferred. When physical components are unavailable, such as with integrated circuit design or specialized lab equipment, a live software simulation is an acceptable entry.

RECOMMENDED BASELINE

- A working hardware demonstration or a live simulation.
- A clear statement explaining the intended user and which theme the project addresses.

Simulation Bench Track Option: Teams unable to assemble physical hardware on-site are fully eligible to enter via **live software simulations** using specialized simulation environments or CAD platforms, paired with live functional demonstrations, telemetry logs, and component validation schematics.

TRACK 02 · UI / UX

AI Interfaces · Accessibility · Human-Centred Redesign

The Process Everyone Dreads

02.1

COVERS: Human-centred redesign, government portal redesign, citizen services, accessibility, forms, approvals, queues

- Most citizen portals, campuses, and municipal services rely on digital processes that frustrate everyday users: clunky public portals that session out mid-application, bureaucratic forms requesting redundant certificates, confusing KYC or identity verification steps, slow multi-step approvals, or physical queues that could be replaced by real-time status updates. These are user-experience failures where systems were designed around administrative rules rather than the people using them.
- Select one administrative, government, or civic process that people frequently dread or struggle to complete (such as public welfare portals, citizen grievance systems, transit pass renewals, municipal certificate requests, student hostel/scholarship workflows, or document verification). Observe at least three people attempting it, document where friction occurs, and redesign the journey from initiation to confirmation. Existing administrative constraints remain: approvals still require verification and payments must be confirmed.
- From redesign, focus on visual flow, status visibility, and information architecture. From accessibility, ensure the redesign supports users currently excluded: screen-reader users, people with low vision, non-English speakers, or users on slow cellular networks. From AI interfaces, identify routine steps a model could draft while keeping the user in control of approvals.

RECOMMENDED BASELINE

- A clickable prototype in Figma or working frontend code covering the redesigned user journey (campus portal, government service, or civic workflow).
- A clear walkthrough showing the original friction points and how the redesign simplifies them.

The Living Canvas: Human-AI Collaboration

02.2

COVERS: AI interfaces, human-AI collaboration, canvas interactions, cognitive accessibility, tool redesign

- Most current AI interfaces force work into a chat sidebar or trigger unpredictable auto-completions that interrupt typing. When people work on complex tasks, such as architectural drafting, code structuring, video editing, or research synthesis, work takes place across a visual canvas. Confining generative tools to text boxes disconnects AI assistance from the user's primary workspace.
- Select a complex professional or creative task that currently requires navigating dense toolbars and nested menus. Design an interface where AI suggestions appear directly on the canvas as visual layers, alternative branches, or inline edits. The AI backend can be simulated or connected to a live API; the core challenge is giving users intuitive controls to review, adjust, or discard machine suggestions without losing their working state.
- From AI interfaces, create direct canvas controls, visual branching, and instant rollback. From accessibility, reduce cognitive load through clean hierarchy, customizable density for neurodivergent users, and full keyboard navigation. From redesign, simplify an intimidating legacy software interface into an adaptable collaborative workspace.

RECOMMENDED BASELINE

- A clickable prototype showing an end-to-end task completed on the canvas with AI assistance.
- Visual states for human editing, reviewing an AI suggestion, and accepting or rejecting modifications.

TRACK 03 · GEN AI & ML

Computer Vision & Detection · Applied ML · Neural Optimization & Agents

Vision in the Wild: Applied Machine Learning & Detection

03.1

COVERS: Computer vision, object detection, YOLO, CNN classification, healthcare diagnostics, civic safety, edge inference

- Many machine learning projects stop at a high accuracy score inside an isolated Jupyter notebook. When these models face real operating conditions, they encounter challenges that benchmark datasets hide: poor lighting in clinic examination rooms, camera shake on moving vehicles, partial occlusions on factory floors, dusty lenses, or noisy backgrounds. In practical settings, an algorithm that flags false alarms or misses subtle edge cases creates user fatigue or dangerous blind spots. The real test of an applied machine learning model is taking detection or classification architectures and building dependable tools that solve authentic problems under messy conditions.
- Select a tangible detection, classification, or screening problem and build a complete working machine learning tool around it. Teams are strongly urged to focus on applying these techniques to a novel, creative, and genuinely impactful problem rather than standard toy tutorials. In healthcare and diagnostics, screen chest X-rays, identify lung CT nodules, classify skin lesions, or assist visually impaired users with obstacle awareness. In civic safety, detect road potholes, monitor helmet and safety gear compliance, or spot early signs of fire and smoke. In agriculture, identify crop blights and pests from leaf photos. Train or adapt standard architectures such as YOLO (v8, v11), MobileNet, ResNet, EfficientNet, or custom convolutional networks on free cloud environments like [Google Colab](#) or [Kaggle](#). Teams are encouraged to consider building a software system around their model (such as an interactive web application, a mobile interface, a Streamlit/Gradio dashboard, or an integrated camera tool) so non-technical users can interact with and test predictions in real time, rather than presenting only a standalone script or raw notebook.
- From computer vision, measure inference latency, frame rates, and precision-recall trade-offs across standard hardware. From applied ML, implement techniques that improve reliability on real inputs: data augmentations simulating blur and lighting variations, confidence score calibration, or explainability maps (such as Grad-CAM) showing where the network focused its attention. From system integration, engineer the user-facing software so predictions translate into meaningful real-world utility: exportable diagnostic reports, live visual overlays, alert notifications, or localized geographic incident markers.

RECOMMENDED BASELINE

- A working detection or classification vision model solving a creative, impactful problem.
- A working demonstration or interactive interface (such as a web app, mobile app, or dashboard) allowing live prediction testing.
- Demonstrated on sample test images, video, or live camera feed showing predicted labels, confidence scores, and system actions.

The Learned Loop: From Search to Learning

03.2

COVERS: Learned heuristics, neural surrogates, reinforcement learning, optimization algorithms, classical baselines, speedup

- Many traditional software systems solve complex optimization problems using slow trial-and-error, rigid rule tables, or brute-force search routines. Everyday examples include delivery route planning, server load scheduling, game move solvers, compiler optimization, or engineering parameter tuning. Because these classical algorithms recalculate decisions from scratch on every single run, they become painfully slow and resource-heavy as problem complexity grows.
- Pick a slow, repetitive, or search-heavy problem normally solved with classical algorithms, heuristic approximations, or manual trial-and-error. Replace or speed up that traditional loop with a machine learning model—such as a neural network surrogate, a reinforcement learning agent, or an iterative learning heuristic. Demonstrate that your learned system either produces decisions in a fraction of the time, or discovers higher-performing configurations that standard rule-based passes missed.
- From agentic & ML workflows, build a closed feedback loop where your model proposes candidate solutions, evaluates them against a simulator or scoring function, and refines its strategy across iterations. From smart caching, remember past high-performing states to guide new searches instead of starting from scratch. From on-device models, run the trained model locally so evaluations complete in milliseconds without remote APIs. From classical ML, run direct head-to-head benchmarks against the traditional algorithm or heuristic baseline on speed, iteration count, and solution quality.

RECOMMENDED BASELINE

- A working model or script generating candidate decisions, configurations, or design suggestions.
- A comparison against a traditional baseline algorithm or heuristic on a sample problem.
- Basic metrics comparing speed or solution quality between the two methods.

TRACK 04 · OFFICIAL BRIEFS

TRACK 04 · CYBERSECURITY

FinTech · Fraud Shield · Zero-Trust Cybersecurity

The Scam That Almost Worked

04.1

COVERS: Fraud detection, social engineering, UPI payments, zero-trust verification, user awareness

- Social engineering attacks targeting Indian digital payment users succeed because they exploit psychological urgency rather than technical bugs: fake UPI collect requests, fraudulent utility bill notices, impersonation calls, deceptive job deposits, and deceptive APK links. Victims make decisions under artificial pressure before they can verify the source.
- Select one active fraud vector observed in your community. Analyze the attack progression: initial contact, psychological trigger, data requested, and the transaction moment. Build a defensive tool that detects the threat or interrupts the attack sequence before funds transfer. Work exclusively on test environments, simulated transactions, or sanitized data.
- From fraud detection, classify suspicious patterns across sender metadata, message text, payment links, and QR payloads. From FinTech, design payment authorization flows that highlight inconsistencies without disrupting legitimate transactions. From zero-trust, assume credentials or OTPs have been compromised and require secondary contextual verification. Interactive tools that train users to recognize active scam patterns in safe simulations are also eligible.

RECOMMENDED BASELINE

- A clear breakdown of the chosen scam pattern (such as fake UPI requests, utility bill scams, or phishing links) with sample messages.
- A working defensive tool, browser extension, or verification script demonstrated live on test data.
- Tested on both suspicious and legitimate sample inputs to show how it flags threats.

Shipped Fast, Left Open

04.2

COVERS: Application security, zero-trust architecture, authentication, payment integration, developer tooling

- Modern software development emphasizes rapid deployment, often leaving routine security oversights in production code: hardcoded API secrets, unauthenticated database endpoints, missing authorization checks between users, and client-side payment values that can be tampered with in transit.
- Build a tool or architectural pattern that prevents or identifies common security weaknesses in rapidly developed web and mobile applications. You may develop a static scanner, a pre-commit review tool, or build a hardened reference application demonstrating mitigations against specific vulnerabilities. Test exclusively on self-hosted environments or deliberately vulnerable applications.
- From zero-trust, enforce continuous request validation, strict role-based access control, and least-privilege database permissions. From FinTech, secure payment callbacks, webhook signatures, and transactional integrity checks. From fraud detection, identify automated abuse including credential stuffing, fake registrations, and coupon exploitation through anomaly detection in application logs.

RECOMMENDED BASELINE

- A demo application or test environment showing common security oversights (such as exposed secrets, broken access rules, or injection).
- A working scanner or defensive pattern that catches or blocks these vulnerabilities live.
- A brief explanation of each flaw, how it could be exploited, and how the fix secures it.

| TRACK 05 · OFFICIAL BRIEFS

TRACK 05 · SUSTAINABLE LIVING

Climate Tech · Corporate Sustainability · Resource Efficiency · Smart Infrastructure

When the Grid Trembles: Carbon-Aware Infrastructure

05.1

COVERS: Climate tech, smart energy, demand response, carbon-aware computing, smart infrastructure, resource efficiency

- Renewable power generation is constrained by mismatches between supply and demand. Solar generation peaks at midday when commercial electricity demand is moderate, while fossil-fueled peaker plants activate during evening demand spikes. Simultaneously, server clusters, campus cooling plants, and electric vehicle chargers draw power continuously without factoring in real-time grid carbon intensity.
- Build software that coordinates electricity-intensive operations with clean energy availability. Schedule computational workloads, battery storage cycles, cold storage chilling, or electric vehicle charging during hours of peak renewable generation and low carbon intensity. Public regional grid emission data, municipal power logs, and synthetic workload traces provide sufficient inputs without needing physical hardware.
- From smart infrastructure, develop automated load-scheduling algorithms for vehicle charging depots, campus facilities, or cloud compute instances. From climate tech, forecast regional grid carbon intensity from open power data to schedule low-carbon execution windows. From resource efficiency, optimize battery degradation cycles and peak-load reduction. From corporate sustainability, produce audit-ready records quantifying verifiable emissions avoided through dynamic scheduling.

RECOMMENDED BASELINE

- A working software prototype that takes power or carbon data and schedules tasks into greener windows.
- A simple comparison showing estimated energy savings or carbon reductions compared to an unscheduled baseline.

Hyperlocal Climate Resilience: Heat, Drought, and Weather Extremes

05.2

COVERS: Climate tech, heat waves, drought monitoring, weather telemetry, geospatial AI, flood prediction, microclimates

- While macro climate models forecast broad regional averages, environmental hazards strike at the neighborhood, field, and street level. Communities face diverse, localized environmental pressures: concrete urban heat islands that elevate outdoor temperatures, prolonged droughts that deplete soil moisture and groundwater tables, dust storms and air quality inversions, or sudden cloudbursts and catchment overflows like the recent devastating floods in Nepal. In each case, communities and local administrators lack the localized situational telemetry needed to anticipate these conditions.
- Build a software tool or early warning system that converts open geospatial datasets, satellite remote sensing, weather radar, soil telemetry, or topographic maps into localized resilience intelligence. You can develop systems to predict, detect, or track any environmental threat: forecasting dangerous heat stress for outdoor workers, detecting early signs of crop drought and water stress, predicting flash flood runoff and slope instability before downpours strike, or mapping urban microclimates and air quality hazards. The goal is giving residents, farmers, or municipal teams actionable lead time to respond.
- From climate tech, fuse multispectral satellite feeds (such as Sentinel, Landsat, or NASA GPM) with local topography to track vegetation health, thermal bands, soil moisture deficits, or runoff accumulation. From smart infrastructure, route pedestrians and field logistics through cooler or safer corridors, automate drainage and pump schedules, or model district cooling and shading interventions. From resource efficiency, estimate rainwater catchment potential, map agricultural groundwater depletion, or model drought-resistant irrigation needs. From corporate sustainability, assess climate asset risk to help campuses, supply chains, and municipal services prepare for recurring weather disruptions.

RECOMMENDED BASELINE

- A functioning software tool or map that tracks a localized environmental hazard (such as heat, drought, flood risk, or weather changes) for a specific district or campus.
- Provides actionable alerts or recommendations, such as safe routes, cooling points, or evacuation lead time.